Spam Detection Using Naive Bayes: A Complete Project Overview

Asal Yazdanpanah
Student ID: 610002071

January 30, 2025

Introduction

This project focuses on developing a spam detection system using the Naive Bayes algorithm, a probabilistic classifier widely used in text classification tasks. The primary objective is to classify messages as either spam or ham (non-spam) based on their content. The workflow includes data preprocessing, manual implementation of the Naive Bayes algorithm, model evaluation, and a performance comparison with Scikit-learn's built-in implementation.

Project Workflow

The project was executed in the following key steps:

1. Data Preprocessing

The dataset consisted of labeled messages (spam or ham). The following preprocessing steps were applied:

- **Text Cleaning:** Removed special characters, punctuation, and stopwords to focus on meaningful words.
- **Tokenization:** Split each message into individual words (tokens).
- **Bag-of-Words Representation:** Created a vocabulary of unique words and converted each message into a numerical vector based on word frequencies.

2. Manual Naive Bayes Implementation

The Naive Bayes algorithm was implemented from scratch using the following steps:

1. **Prior Probability (P(c)):** Calculated the proportion of messages belonging to each class (spam or ham).
2. **Likelihood (P(w—c)):** Estimated the probability of each word in the vocabulary given a class using Laplace smoothing to handle unseen words.

3. **Posterior Probability (P(c—X)):** Computed the posterior probability for each test message and classified the message into the class with the highest probability.

The posterior probability was calculated using the formula:

$$P(c|X) \propto P(c) \cdot \prod_{i=1}^{n} P(w_i|c),$$

where w_i represents words in the message, and n is the total number of words.

3. Scikit-learn Naive Bayes Implementation

The built-in `MultinomialNB` from Scikit-learn was used to train and test the model on the same data. While the underlying calculations are identical to the manual implementation, Scikit-learn's version is optimized for efficiency, making it faster and more suitable for large datasets.

4. Model Evaluation

Both models were evaluated using the following metrics:

- **Accuracy:** Proportion of correctly classified messages.
- **Precision:** Proportion of messages predicted as spam that are actually spam.
- **Recall:** Proportion of actual spam messages correctly identified as spam.
- **F1-Score:** Harmonic mean of precision and recall.

Results and Discussion

The performance metrics for both implementations are summarized in Table 1.

Metric	Manual Naive Bayes	Scikit-learn Naive Bayes
Accuracy	0.95	0.95
Precision	0.96	0.96
Recall	0.93	0.93
F1-Score	0.94	0.94

Table 1: Performance Comparison of Manual and Scikit-learn Naive Bayes Models

Key Observations

- Both implementations achieved identical performance metrics, validating the correctness of the manual implementation.
- The Scikit-learn implementation is faster and more efficient, making it more suitable for large datasets.
- The manual implementation provided a deeper understanding of the underlying mechanics of the Naive Bayes algorithm, including prior probabilities, likelihoods, and Laplace smoothing.

Conclusion

This project successfully implemented and evaluated the Naive Bayes algorithm for spam detection. The manual implementation performed equally well compared to Scikit-learn's optimized version, achieving high accuracy, precision, recall, and F1-scores. While Scikit-learn is more efficient, the manual implementation reinforced fundamental concepts such as prior probabilities, likelihoods, and Laplace smoothing. This balance of theory and practice highlights the effectiveness and educational value of the project.